

Padrões de Criação

Cenário

Você foi contratado para desenvolver um sistema de E-commerce.

O sistema precisa:

- criar pedidos
- processar pagamentos
- gerar configurações por ambiente
- duplicar produtos
- manter configurações globais

Vamos aplicar os Padrões Criacionais em um caso real.

Singleton — Configuração Global

Problema

A aplicação precisa carregar configurações apenas uma vez.

Exemplo:

- URL do banco
- token API
- ambiente

Singleton — Configuração Global

Java

```
public class AppConfig {  
  
    private static AppConfig instance;  
  
    private String environment;  
  
    private AppConfig() {  
        environment = "PRODUCAO";  
    }  
  
    public static AppConfig getInstance() {  
  
        if(instance == null) {  
            instance = new AppConfig();  
        }  
  
        return instance;  
    }  
  
    public String getEnvironment() {  
        return environment;  
    }  
}
```

Java

```
AppConfig config = AppConfig.getInstance();  
  
System.out.println(config.getEnvironment());
```

Factory Method — Pagamentos

Problema

O sistema aceita:

- PIX
- Cartão
- Boleto

Evitar vários new.

-

Java

```
interface Payment {  
    void process();  
}
```

Java

```
class PixPayment implements Payment {  
  
    public void process() {  
        System.out.println("Pagamento PIX");  
    }  
}
```

Java

```
class CreditCardPayment implements Payment {  
  
    public void process() {  
        System.out.println("Pagamento Cartão");  
    }  
}
```

Factory Method — Pagamentos

Java

```
class PaymentFactory {  
  
    public static Payment create(String type) {  
  
        switch(type) {  
  
            case "PIX":  
                return new PixPayment();  
  
            case "CARD":  
                return new CreditCardPayment();  
  
            default:  
                throw new IllegalArgumentException();  
        }  
    }  
}
```

Java

```
Payment payment = PaymentFactory.create("PIX");  
  
payment.process();
```

Factory Method — Pagamentos

Problema

Pedido possui muitos atributos:

- cliente
- endereço
- cupom
- entrega
- observação
- desconto

Construtores gigantes ficam ilegíveis.

```
Java

public class Order {

    private String customer;
    private String address;
    private double discount;

    private Order(OrderBuilder builder) {
        this.customer = builder.customer;
        this.address = builder.address;
        this.discount = builder.discount;
    }

    public static class OrderBuilder {

        private String customer;
        private String address;
        private double discount;

        public OrderBuilder customer(String customer) {
            this.customer = customer;
            return this;
        }

        public OrderBuilder address(String address) {
            this.address = address;
            return this;
        }

        public OrderBuilder discount(double discount) {
            this.discount = discount;
            return this;
        }

        public Order build() {
            return new Order(this);
        }
    }
}
```

```
Java

Order order = new Order.OrderBuilder()
    .customer("Maria")
    .address("Recife")
    .discount(10)
    .build();
```